MODUL PELATIHAN • HARI 1 • MATERI 1, 2, DAN 3

Building modern web application

Dari pengenalan Node.js hingga halaman dinamis dengan EJS.

01 **Pengenalan Node.js dan web modern** — Client-server, teknologi web, Node.js, Express, dan Lab 1 — 02-14

02 **Persiapan lingkungan kerja** — Node.js, VS Code, Git, proyek Express, dan Lab 2 — 15-28

03 **Templating dengan EJS** — Template, sintaks EJS, alur data, dan Lab 3 — 29-42

Daftar isi dapat diklik. Semua pilihan simulasi disajikan sebagai halaman terpisah.

HARI 1 • MATERI 1

Pengenalan Node.js dan web modern

Memahami cara kerja komunikasi *client-server*, tumpukan teknologi web modern, serta peran Node.js dan Express di sisi server.

Kode: TX-BE01 **45 menit materi • 25 menit Lab 1**

POKOK BAHASAN SESI INI

01 Konsep dasar situs web & *client-server*

02 Tumpukan teknologi: *frontend* & *backend*

03 Peran Node.js dan Express.js

04 Perbedaan Node.js dengan PHP

05 Kapan menggunakan Node.js & Lab 1

PETA BELAJAR

Lima bahasan utama sesi ini

Fondasi yang perlu kita kuasai sebelum membangun server aplikasi web.

01 Konsep dasar situs web

Pengertian situs web dan cara peramban berkomunikasi dengan server.

02 *Tech stack* modern

Pembagian tugas antara sisi tampilan (*frontend*) dan sisi pengolah data (*backend*).

03 Node.js dan Express.js

Menjalankan JavaScript di luar peramban dan merangkai server web dengan cepat.

04 Node.js vs PHP

Perbandingan bahasa, cara kerja *asynchronous* vs *synchronous*, dan ekosistem NPM.

05 Kapan memilih Node.js & praktik Lab 1

Memilih skenario aplikasi yang tepat lalu mengeksekusi kode JavaScript langsung di terminal lewat REPL Node.js.

01 • KONSEP DASAR WEBSITE

Apa itu situs web dan bagaimana cara kerjanya?

Situs web adalah kumpulan halaman di internet yang diakses menggunakan peramban (*web browser*) seperti Google Chrome atau Mozilla Firefox melalui pola **Client-Server**.

SISI PENGGUNA

Client (Klien)

Perangkat dan peramban yang digunakan pengguna untuk meminta halaman atau mengirim data.

1. Permintaan (*Request*) →

Membuka alamat web atau mengirim formulir

← **2. Jawaban (*Response*)**

Mengirimkan halaman HTML atau data JSON

SISI PENYEDIA

Server

Komputer jarak jauh yang menyimpan data web, memproses aturan aplikasi, dan mengirim hasilnya kembali ke klien.

Komunikasi web selalu berputar pada siklus: **klien meminta, server melayani**.

02 • TECH STACK MODERN

Dua bagian utama pembentuk aplikasi web

Untuk membangun sebuah situs web yang utuh, kita memerlukan *tech stack* (tumpukan teknologi) yang terbagi menjadi dua sisi:

SISI KLIEN (PERAMBAN)

Frontend

Segala sesuatu yang tampil di layar dan berinteraksi langsung dengan pengguna.

- Menampilkan teks, gambar, tabel, dan formulir.
- Merapikan tata letak, warna, dan ukuran tombol.
- Merespons klik, ketikan, serta animasi di layar.

HTML **CSS** **JavaScript**

SISI SERVER (DI BALIK LAYAR)

Backend

Mesin di balik layar yang mengolah logika aplikasi, keamanan, dan penyimpanan data.

- Memvalidasi data dan memeriksa hak akses akun.
- Menjalankan perhitungan serta aturan bisnis aplikasi.
- Membaca dan menyimpan data ke *database*.

Node.js **Express** **PostgreSQL**

02 • TECH STACK MODERN • FRONTEND

Tiga pilar utama di sisi *frontend*

Setiap halaman yang tampil di peramban dibangun oleh kerja sama tiga teknologi ini:

01 STRUKTUR KONTEN

HTML

Menyusun kerangka dasar halaman web, mulai dari judul, paragraf, daftar, tabel, hingga kolom input formulir.

> Ibarat fondasi, dinding, dan pintu pada sebuah gedung.

02 GAYA & TATA LETAK

CSS

Mengatur tampilan visual seperti warna, jenis huruf, jarak antarelemen, dan kerapian layar di komputer maupun ponsel.

> Ibarat warna cat, pencahayaan, dan penataan interior.

03 INTERAKTIVITAS

JavaScript

Menghidupkan halaman agar bisa merespons tindakan pengguna: menampilkan peringatan, validasi formulir, dan animasi.

> Ibarat instalasi listrik dan pintu otomatis di dalam gedung.

02 • TECH STACK MODERN • BACKEND

Komponen penggerak di sisi *backend*

Saat pengguna mengirim data atau meminta informasi, *backend* memprosesnya melalui dua lapisan utama:

LAPISAN BACKEND	FUNGSI UTAMA	CONTOH TEKNOLOGI
Bahasa & Runtime Server	Menerima permintaan dari peramban, menjalankan aturan bisnis, menjaga keamanan, dan menyusun jawaban.	**Node.js (JavaScript)**, PHP, Python, Java
Database (Basis Data)	Menyimpan, mencari, dan memperbarui data secara permanen agar tidak hilang saat server dimatikan.	**PostgreSQL**, MySQL, MongoDB

Fokus Pelatihan TX-BE01: Kita menggunakan **Node.js & Express.js** untuk memproses logika di sisi server, serta **PostgreSQL** untuk menyimpan data aplikasi.

03 • NODE.JS & EXPRESS

Apa itu Node.js?

Node.js adalah *runtime environment* (lingkungan eksekusi) yang memungkinkan kita menjalankan kode JavaScript di luar peramban, tepatnya di sisi server.

DULU (SEBELUM NODE.JS)

JavaScript terbatas di peramban

JavaScript hanya bisa dipakai untuk mengatur tampilan di sisi klien. Untuk membuat program di server, pengembang harus mempelajari bahasa lain seperti PHP atau Java.

SEKARANG (DENGAN NODE.JS)

JavaScript berjalan di semua sisi

Kode JavaScript dapat dieksekusi langsung di komputer server. Kita bisa mengembangkan aplikasi secara ***Full-stack*** (*Frontend* dan *Backend*) cukup dengan satu bahasa.

> Bahasanya tetap **JavaScript**; **Node.js** adalah mesin yang menjalankannya di server.

03 • NODE.JS & EXPRESS

Apa itu Express.js?

Express.js adalah kerangka kerja (*framework*) minimalis dan fleksibel untuk Node.js yang mempermudah pembuatan server web dan API.

01 Lebih cepat dan ringkas

Menyederhanakan penulisan kode dibanding menggunakan modul bawaan Node.js murni dari nol.

02 Pengaturan rute (*routing*) rapi

Memudahkan kita menentukan halaman atau data apa yang dikirim saat pengguna membuka alamat tertentu.

03 Siap untuk halaman web & API

Cocok untuk mengirim tampilan HTML (lewat EJS) maupun data JSON untuk aplikasi lain.

CONTOH RINGKAS SERVER EXPRESS.JS

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Halo dari Express!');
});

app.listen(3000);
```

Hanya dengan beberapa baris kode, server web sudah siap menerima permintaan di port 3000.

04 • PERBEDAAN NODE.JS DENGAN PHP

Membandingkan Node.js dengan PHP

Banyak situs web dibangun menggunakan PHP. Berikut tiga perbedaan utamanya dibandingkan dengan Node.js:

ASPEK	NODE.JS	PHP (TRADISIONAL)
Bahasa Pemrograman	Menggunakan **JavaScript**, sehingga satu bahasa bisa dipakai di *frontend* dan *backend*.	Menggunakan bahasa **PHP** yang dirancang khusus untuk pemrograman web di server.
Arsitektur Eksekusi	**Asynchronous / Non-blocking**: dapat menangani ribuan permintaan bersamaan tanpa saling tunggu.	**Synchronous / Blocking**: menunggu satu tugas tuntas sebelum lanjut ke tugas berikutnya.
Ekosistem Pustaka	Mengandalkan **NPM** (*Node Package Manager*), repositori pustaka *open-source* terbesar di dunia.	Mengandalkan **Composer** untuk memasang dan mengelola pustaka tambahan.

04 • ARSITEKTUR EKSEKUSI

Ilustrasi: *Synchronous* vs *Asynchronous*

Node.js: Asynchronous (Non-blocking) | Tradisional: Synchronous (Blocking)

TIDAK SALING MENUNGGU

Server tetap melayani permintaan lain saat menunggu proses data

Ketika Permintaan A sedang menunggu pembacaan database, Node.js langsung menerima dan memproses Permintaan B serta C. Begitu data A selesai diambil, hasilnya langsung dikirimkan kembali.



Analogi kasir: setelah mencatat pesanan pelanggan pertama, kasir langsung melayani pelanggan berikutnya tanpa berdiri diam menunggu dapur selesai memasak.

04 • ARSITEKTUR EKSEKUSI

Ilustrasi: *Synchronous* vs *Asynchronous*

Node.js: Asynchronous (Non-blocking) | **Tradisional: Synchronous (Blocking)**

MENUNGGU GILIRAN SELESAI

Satu tugas harus tuntas sebelum proses lanjut ke tugas berikutnya

Pada model synchronous tradisional, saat Permintaan A menunggu pembacaan database, proses pekerja tersebut tertahan hingga selesai sebelum dapat beralih menangani antrean berikutnya.



Analogi kasir: setelah mencatat pesanan pelanggan pertama, kasir langsung melayani pelanggan berikutnya tanpa berdiri diam menunggu dapur selesai memasak.

05 • KAPAN MENGGUNAKAN NODE.JS?

Empat skenario terbaik untuk memilih Node.js

Gunakan Node.js saat aplikasi yang Anda bangun memiliki kebutuhan berikut:

01

Aplikasi *real-time*

Membangun aplikasi percakapan langsung (*chat*), notifikasi langsung (*live notification*), kolaborasi dokumen, atau permainan multipemain.

02

Layanan API intensitas tinggi

Membangun server API yang melayani pertukaran data berformat **JSON** dalam jumlah banyak dan frekuensi tinggi.

03

Satu bahasa di semua sisi

Menggunakan **JavaScript** secara seragam di sisi *frontend* maupun *backend* sehingga kerja tim lebih selaras.

04

SPA & aplikasi web modern

Membangun *Single Page Application* (SPA) atau dasbor aplikasi web dinamis yang membutuhkan waktu respons cepat.

SESI PRAKTIK • LAB 1 (25 MENIT)

Saatnya praktik: menjalankan JavaScript di terminal

Ikuti langkah **Lab 1: Eksplorasi Node.js Dasar** tanpa menggunakan peramban:

1. Buka **Terminal** (macOS/Linux) atau **Command Prompt / PowerShell** (Windows).
2. Ketik perintah `node` lalu tekan **Enter** untuk masuk ke mode interaktif **REPL** (*Read-Eval-Print-Loop*).
3. Ketik dan jalankan kode JavaScript berikut di dalam terminal:

```
let pesan = "Halo dari Server Node.js!";
console.log(pesan);

let hasil = 10 * 5;
console.log("Hasil perkalian: " + hasil);
```

Tekan `Ctrl + C` sebanyak dua kali untuk keluar dari mode REPL.

TARGET LAB 1

JavaScript di luar peramban

Membuktikan bahwa JavaScript kini dapat dieksekusi langsung di sistem operasi komputer (server) untuk melakukan komputasi dan tugas lainnya.

SETELAH LAB 1 SELESAI

Kita akan lanjut ke **Materi 2: Persiapan Lingkungan Kerja** (Instalasi Node.js, VS Code, Git, dan inisialisasi proyek Express).

HARI 1 • MATERI 2

Persiapan lingkungan kerja dan proyek Express

Menyiapkan perangkat pengembangan (Node.js, VS Code, dan Git), menginisialisasi proyek baru dengan `npm`, serta memahami struktur folder aplikasi Express.

Kode: TX-BE01 **35 menit materi • 35 menit Lab 2**

POKOK BAHASAN SESI INI

01 Instalasi Node.js versi LTS dan `npm`

02 Editor kode Visual Studio Code (VS Code)

03 Pengenalan kontrol versi dengan Git

04 Membuat proyek Express dari awal

05 Struktur folder standar & praktik Lab 2

02

PETA BELAJAR

Lima langkah menyiapkan lingkungan kerja

Tahapan berurutan mulai dari pemasangan perangkat hingga struktur proyek siap dijalankan.

01 Instalasi Node.js & NPM

Memilih versi LTS yang stabil dari situs resmi `nodejs.org` beserta pengelola paket `npm`.

02 Instalasi VS Code

Menggunakan editor kode ringan dengan penyorot sintaks, *auto-complete*, dan terminal bawaan.

03 Pengenalan Git

Melacak riwayat perubahan kode, kembali ke versi aman saat terjadi galat, dan berkolaborasi di tim.

04 Inisialisasi proyek Express

Membuat konfigurasi `package.json` dengan `npm init` lalu memasang pustaka Express.

05 Struktur folder aplikasi & praktik Lab 2

Mengenal peran `node_modules`, `package.json`, `app.js`, `public/`, dan `views/` lalu menjalankan server di port 3000.

01 • INSTALASI NODE.JS

Memasang Node.js versi LTS di komputer

Sebelum membangun aplikasi dengan Express, kita memasang mesin Node.js secara gratis melalui situs resmi **nodejs.org**.

PILIHAN UTAMA • DISARANKAN

Versi LTS (*Long Term Support*)

Dirancang dengan fokus pada kestabilan dan dukungan pemeliharaan jangka panjang. Sangat aman dipakai untuk pelatihan maupun server produksi.

VERSI EKSPERIMENTAL

Versi *Current* (Terbaru)

Memuat fitur-fitur paling baru yang sedang diuji coba, namun lebih sering mengalami perubahan sehingga kurang ideal untuk fondasi proyek produksi.

Sudah termasuk NPM: Proses instalasi Node.js otomatis menyertakan `npm` (*Node Package Manager*). Periksa keberhasilan instalasi di terminal dengan perintah `node -v` dan `npm -v`.

02 • INSTALASI VS CODE

Editor kode Visual Studio Code

VS Code adalah editor kode gratis dan ringan buatan Microsoft yang sangat cocok dipadukan dengan Node.js.

01 BANTUAN PENULISAN

Penyorot sintaks & *auto-complete*

Memberi warna berbeda pada tiap struktur kode dan melengkapi penulisan perintah secara otomatis agar lebih cepat dan minim salah ketik.

> Memudahkan pembacaan kode JavaScript, HTML, dan JSON.

02 PRAKTIS & MENYATU

Terminal terintegrasi

Dilengkapi terminal bawaan (*integrated terminal*) untuk menjalankan perintah `node` dan `npm` langsung di dalam jendela editor.

> Buka cepat melalui menu **Terminal → New Terminal**.

03 DUKUNGAN EKSTENSI

Ekosistem ekstensi luas

Tersedia ribuan ekstensi tambahan untuk merapikan format kode (Prettier), mendukung template EJS, hingga menguji REST API.

> Dapat dipasang langsung dari panel Extensions di sisi kiri.

03 • PENGENALAN GIT

Mengelola perubahan kode dengan Git

Git adalah sistem pengontrol versi (*Version Control System*) yang menjaga keamanan riwayat kode selama pengembangan aplikasi.

01 RIWAYAT PERUBAHAN

Melacak perubahan berkas kode

Mencatat setiap penambahan dan pengubahan baris kode dari waktu ke waktu secara terperinci.

> Kita tahu kapan sebuah fitur ditambahkan dan bagian mana yang berubah.

02 PEMULIHAN AMAN

Kembali ke versi sebelumnya

Memungkinkan kita memutar balik kode ke kondisi terakhir yang berjalan normal apabila terjadi galat fatal.

> Mencegah hilangnya kode yang sudah berhasil dibuat.

03 KERJA SAMA TIM

Kolaborasi tanpa bentrok kode

Memudahkan beberapa anggota tim mengerjakan proyek yang sama tanpa saling menimpa atau menghapus berkas rekan kerja.

> Fondasi penting sebelum aplikasi diunggah ke GitHub dan Vercel.

04 • MEMBUAT PROYEK EXPRESS DARI AWAL

Alur inisialisasi proyek Node.js dan Express

Pembuatan proyek Node.js selalu dimulai dengan menyiapkan berkas konfigurasi sebelum memasang pustaka tambahan.

LANGKAH 01

Siapkan folder proyek

Buat folder baru di komputer (contoh: `web-app-saya`), lalu buka folder tersebut di dalam VS Code.

LANGKAH 02

Inisialisasi dengan `npm init -y`

Menghasilkan berkas `package.json` secara otomatis untuk mencatat konfigurasi dan identitas proyek.

LANGKAH 03

Pasang Express (`npm install express`)

Mengunduh pustaka Express beserta dependensinya ke dalam folder `node_modules/`.

LANGKAH 04

Buat berkas utama `app.js`

Menulis kode server Express pertama dan menjalankannya melalui terminal menggunakan perintah `node app.js`.

04 • KONFIGURASI PROYEK

Mengenal isi berkas package.json

Berkas `package.json` berfungsi sebagai kartu identitas proyek sekaligus daftar belanja pustaka yang digunakan aplikasi.

01 Identitas proyek

Menyimpan nama aplikasi (`name`), versi (`version`), dan berkas utama yang dijalankan (`main`).

02 Daftar *dependencies*

Mencatat setiap pustaka eksternal yang dipasang lewat `npm install` , seperti `express` .

03 Memudahkan pemindahan proyek

Saat proyek dibuka di komputer lain, cukup jalankan `npm install` untuk mengunduh semua pustaka yang tercatat di sini.

CONTOH ISI `PACKAGE.JSON`

```
{
  "name": "web-app-saya",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error\" && exit 1"
  },
  "dependencies": {
    "express": "^4.21.0"
  }
}
```

Bagian `dependencies` otomatis bertambah saat kita menjalankan `npm install express` .

05 • STRUKTUR FOLDER APLIKASI

Struktur folder standar aplikasi Express

Penataan folder yang rapi memisahkan berkas konfigurasi, logika server, aset statis, dan tampilan halaman:

NAMA FOLDER / BERKAS	PERAN DI DALAM PROYEK	CATATAN PENTING
`node_modules/`	Tempat tersimpannya kode dari pustaka pihak ketiga yang diunduh oleh `npm`.	**Jangan pernah disalin manual**
`package.json`	Menyimpan identitas proyek, skrip perintah, dan daftar dependensi aplikasi.	Dibuat melalui `npm init`
`app.js` / `index.js`	*Entry point* atau berkas utama yang dijalankan untuk menyalakan server.	Dijalankan dengan `node app.js`
`public/`	Menyimpan berkas statis yang diakses langsung oleh peramban (CSS, gambar, ikon).	Folder aset tampilan
`views/`	Menyimpan berkas template halaman HTML dinamis (seperti berkas EJS).	Digunakan mulai Materi 3

05 • KERAPIAN PROYEK

Cara mengelola folder `node_modules` dengan benar

Folder `node_modules/` dibuat otomatis oleh `npm` dan berisi ribuan berkas pustaka pendukung.

HINDARI CARA INI

Menyalin `node_modules` manual

Memindahkan folder `node_modules` secara langsung saat berbagi proyek ke rekan tim atau mengunggah ke Git.

- Ukuran folder besar dan memuat ribuan berkas kecil.
- Proses salin berkas menjadi sangat lambat.
- Rawan tidak cocok ketika berpindah sistem operasi (misal dari Windows ke macOS/Linux).

PRAKTIK STANDAR INDUSTRI

Gunakan `package.json` & `npm install`

Cukup bagikan kode sumber proyek beserta berkas `package.json` tanpa menyertakan folder `node_modules`.

- Masukkan nama `node_modules/` ke dalam berkas `.gitignore`.
- Ukuran proyek tetap ringan dan bersih saat disimpan di Git.
- Di komputer tujuan, jalankan perintah `npm install` untuk membentuk ulang `node_modules`.

SIMULASI TERMINAL • PERSIAPAN PROYEK

Tiga perintah utama saat membangun proyek Express

1. npm init -y
2. npm install express
3. node app.js

TAHAP 1 • INISIALISASI

Membuat berkas konfigurasi package.json

Perintah ini menyiapkan folder biasa menjadi proyek Node.js dengan membuat berkas package.json berisi pengaturan awal secara otomatis.

TERMINAL DI FOLDER WEB-APP-SAYA

```
$ npm init -y
Wrote to /web-app-saya/package.json:
{
  "name": "web-app-saya",
  "version": "1.0.0",
  "main": "index.js"
}
```

Berkas baru terbentuk: `package.json`

SIMULASI TERMINAL • PERSIAPAN PROYEK

Tiga perintah utama saat membangun proyek Express

1. npm init -y
2. **npm install express**
3. node app.js

TAHAP 2 • INSTALASI PUSTAKA

Mengunduh kerangka kerja Express ke dalam proyek

NPM mengunduh paket Express beserta pustaka pendukungnya ke dalam folder node_modules dan mencatatnya pada bagian dependencies di package.json.

TERMINAL DI FOLDER WEB-APP-SAYA

```
$ npm install express

added 64 packages, and audited 65 packages in 2s
found 0 vulnerabilities
```

Terbentuk: folder `node_modules/` dan pembaruan `package.json`

SIMULASI TERMINAL • PERSIAPAN PROYEK

Tiga perintah utama saat membangun proyek Express

1. npm init -y
2. npm install express
3. node app.js

TAHAP 3 • MENJALANKAN SERVER

Menyalakan aplikasi server di port 3000

Node.js mengeksekusi berkas app.js. Server kini aktif mendengarkan permintaan HTTP di alamat lokal komputer Anda.

TERMINAL DI FOLDER WEB-APP-SAYA

```
$ node app.js
Server berjalan di http://localhost:3000
```

Buka `http://localhost:3000` di peramban untuk menguji respons server.

PERSIAPAN LAB 2 • BEDAH KODE

Memahami isi berkas `app.js`

Hanya dengan beberapa baris kode di `app.js`, kita sudah dapat menjalankan server web lokal:

BERKAS `APP.JS`

```
const express = require('express');
const app = express();
const port = 3000;

// Routing dasar
app.get('/', (req, res) => {
  res.send('Hello World! Server Express sudah berjalan.');
});

// Menjalankan server
app.listen(port, () => {
  console.log(`Server berjalan di http://localhost:${port}`);
});
```

01 **Inisialisasi Express & Port**

Memanggil modul `express`, membentuk objek aplikasi `app`, dan menetapkan port `3000`.

02 **Routing dasar (`app.get`)**

Saat peramban membuka alamat utama `/`, server mengirim balasan teks melalui `res.send(...)`.

03 **Menyalakan server (`app.listen`)**

Mengaktifkan server agar mendengarkan permintaan pada `http://localhost:3000`.

SESI PRAKTIK • LAB 2 (35 MENIT)

Saatnya praktik: membuat server Express pertama

Ikuti langkah **Lab 2: Setup Project Express** di komputer Anda:

1. Buat folder baru bernama `web-app-saya` lalu buka folder tersebut di **VS Code**.
2. Buka terminal terintegrasi (`Terminal → New Terminal`), lalu jalankan perintah berikut:

```
npm init -y
npm install express
```

3. Buat berkas `app.js` , ketik kode server Express dari slide sebelumnya, lalu jalankan:

```
node app.js
```

Buka peramban dan akses `http://localhost:3000` untuk melihat pesan **Hello World! Server Express sudah berjalan.**

TARGET LAB 2

Server lokal aktif di port 3000

Peserta berhasil menginisialisasi proyek Node.js, memasang Express, dan menampilkan respons teks dari server lokal di peramban.

SETELAH LAB 2 SELESAI

Kita akan lanjut ke **Materi 3: Templating dengan EJS** untuk merender halaman HTML dinamis dan mengirimkan variabel dari server.

HARI 1 • MATERI 3

Templating halaman dinamis dengan EJS

Menyusun tampilan HTML yang dapat disisipi variabel data, percabangan, dan komponen berulang dari server Express.

Kode: TX-BE01 **Materi 3 & Praktik Lab 3**

POKOK BAHASAN SESI INI

01 Apa itu *template engine* dan peran EJS

02 Alur perubahan berkas EJS menjadi HTML

03 Tiga sintaks tag dasar di dalam EJS

04 Membuat layout dasar dan komponen berulang

05 Menampilkan halaman dengan `res.render()` & Lab 3

PETA BELAJAR

Lima bahasan utama sesi ini

Langkah demi langkah membangun halaman HTML yang dinamis dari sisi server.

01 **Konsep *template engine***

Mengatasi keterbatasan HTML statis dengan menyisipkan data JavaScript dari server.

02 **Alur kerja proses *render***

Cara Express menggabungkan berkas `.ejs` dan objek data menjadi HTML siap tampil.

03 **Sintaks tag EJS**

Mencetak nilai variabel, menjalankan pengulangan (*loop*), dan percabangan (*if-else*).

04 **Layout dasar & komponen**

Memecah bagian *Navbar* dan *Footer* menggunakan mekanisme `include`.

05 **Fungsi `res.render()` & praktik Lab 3**

Membaca berkas EJS dari folder `views/` lalu mengirimkannya bersama data ke peramban.

01 • APA ITU TEMPLATE ENGINE?

Mengapa kita memerlukan *template engine*?

Secara bawaan, server Express hanya mengembalikan teks biasa, data JSON, atau berkas HTML statis yang isinya tidak berubah.

HTML STATIS BIASA

Isi halaman selalu tetap

Teks ditulis langsung di dalam berkas HTML. Kita tidak dapat menyisipkan nama pengguna yang sedang masuk atau daftar barang dari *database* secara otomatis.

TEMPLATE ENGINE • EJS

HTML yang diperkaya data server

Memungkinkan kita menulis struktur HTML sambil menyisipkan variabel JavaScript, pengulangan (*loop*), dan percabangan (*if-else*) sebelum dikirim ke peramban.

Di pelatihan ini kita menggunakan **EJS (*Embedded JavaScript templating*)** karena penulisannya sangat dekat dengan HTML standar.

01 • CARA KERJA EJS

Bagaimana Express memproses berkas EJS?

Peramban hanya memahami HTML standar. Express bertugas menerjemahkan berkas EJS menjadi HTML utuh di sisi server:

1. SUMBER DI SERVER

Data + Berkas EJS

Server menyiapkan objek data JavaScript dan membaca kerangka halaman dari folder `views/`.

Proses Render (`res.render`)

Menyisipkan nilai variabel ke dalam tag EJS

Kirim Respons HTTP

Mengirim dokumen HTML yang sudah jadi

2. HASIL DI PERAMBAN

HTML Siap Tampil

Peramban menerima kode HTML bersih tanpa tag EJS lagi, lalu menampilkannya ke layar pengguna.

Proses penggabungan template dan data berlangsung di server sebelum halaman dikirim ke klien.

01 • SINTAKS DASAR EJS

Tiga tag EJS yang paling sering digunakan

EJS menggunakan tanda kurung siku dan persen untuk menandai bagian yang harus diproses oleh server:

01 CETAK VARIABEL

<%= nilai %>

Mencetak isi variabel JavaScript langsung ke dalam elemen HTML. Karakter khusus otomatis diamankan oleh EJS.

Contoh: `<h1>Halo, <%= nama %></h1>`

02 LOGIKA & LOOP

<% kode %>

Menjalankan logika JavaScript seperti percabangan `if-else` atau pengulangan `forEach` tanpa mencetak teks langsung.

Contoh: `<% if (aktif) { %>`

03 KOMPONEN LAYOUT

<%- include() %>

Menyisipkan potongan berkas EJS lain (seperti *Navbar* atau *Footer*) agar tidak perlu ditulis berulang di tiap halaman.

Contoh: `<%- include('navbar') %>`

02 • MEMBUAT LAYOUT DASAR

Memecah tampilan menjadi komponen yang dapat dipakai ulang

Halaman web hampir selalu memiliki bagian yang berulang seperti *Navbar* di bagian atas dan *Footer* di bagian bawah.

01 Cukup dibuat satu kali

Dengan mekanisme `include` pada EJS, kita memecah *Navbar* dan *Footer* menjadi komponen terpisah.

02 Digunakan di banyak halaman

Halaman beranda, profil, maupun laporan cukup memanggil komponen tersebut tanpa menyalin ulang kode HTML.

03 Mudah dipelihara

Perubahan pada satu berkas komponen otomatis memperbarui tampilan di seluruh halaman yang memakainya.

CONTOH PEMAKAIAN `INCLUDE` DI EJS

```
<!DOCTYPE html>
<html lang="id">
<body>
  <%- include('partials/navbar') %>

  <main>
    <h1>Selamat Datang, <%= namaPengguna %>!
</h1>
  </main>

  <%- include('partials/footer') %>
</body>
</html>
```

Kita akan mempraktikkan pemecahan komponen *partials* ini lebih lanjut saat aplikasi memiliki banyak halaman.

03 • MENAMPILKAN HALAMAN DARI SERVER

Tiga langkah menampilkan berkas EJS dari Express

Express menyediakan fungsi `res.render()` untuk membaca berkas dari folder `views/` dan mengubahnya menjadi HTML utuh:

TAHAPAN / KODE	FUNGSI DI DALAM APLIKASI	LOKASI PENULISAN
`npm install ejs`	Memasang pustaka mesin template EJS ke dalam proyek Express kita.	Dijalankan di Terminal
`app.set('view engine', 'ejs')`	Mengatur Express agar menggunakan EJS sebagai pemroses tampilan bawaan.	Di bagian atas `app.js`
`res.render('index', dataUser)`	Membaca berkas `views/index.ejs`, menyisipkan objek `dataUser`, lalu mengirim HTML ke klien.	Di dalam rute `app.get(...)`

Catatan penamaan berkas: Saat memanggil `res.render('index')`, kita cukup menyebutkan nama berkasnya saja tanpa akhiran `.ejs` karena Express otomatis mencarinya di dalam folder `views/`.

03 • METODE RESPONS EXPRESS

Membedakan `res.send()` dengan `res.render()`

Pada Lab 2 kita menggunakan `res.send()`, sedangkan mulai Lab 3 kita menggunakan `res.render()` untuk halaman web:

DIGUNAKAN PADA LAB 2

`res.send(teks)`

Mengirimkan teks, potongan HTML singkat, atau objek secara langsung ke peramban tanpa melalui berkas template.

- Tidak membaca berkas dari dalam folder `views/`.
- Cocok untuk pesan pengujian cepat atau respons sederhana.
- Kurang rapi jika dipakai menulis halaman HTML yang panjang.

Teks langsung **Tanpa folder views**

DIGUNAKAN MULAI LAB 3

`res.render(namaFile, data)`

Membaca berkas template `.ejs` dari folder `views/`, menggabungkannya dengan data, lalu mengirim HTML utuh.

- Memisahkan logika server (`app.js`) dari kode tampilan (`views/`).
- Dapat menerima objek data JavaScript untuk ditampilkan di halaman.
- Menjadi standar untuk menampilkan halaman antarmuka pengguna.

Folder views/ **Data dinamis** **HTML utuh**

SIMULASI INTERAKTIF • PROSES RENDER EJS

Ilustrasi perubahan kode EJS menjadi HTML

1. Variabel (<%= %>) | 2. Percabangan (if-else) | 3. Pengulangan (loop)

KODE DI VIEWS/INDEX.EJS + DATA SERVER

```
// Data dari app.js:
// { namaPengguna: 'Peserta Pelatihan' }

<h1>Selamat Datang, <%= namaPengguna %>!</h1>
<p>Ini adalah aplikasi pertama menggunakan EJS.</p>
```

HASIL HTML YANG DITERIMA PERAMBAN

```
<h1>Selamat Datang, Peserta Pelatihan!</h1>
<p>Ini adalah aplikasi pertama menggunakan EJS.</p>
```

Tag `<%= namaPengguna %>` diganti dengan isi variabel dari server.

SIMULASI INTERAKTIF • PROSES RENDER EJS

Ilustrasi perubahan kode EJS menjadi HTML

1. Variabel (<%= %>) | **2. Percabangan (if-else)** | 3. Pengulangan (loop)

KODE DI VIEWS/INDEX.EJS + DATA SERVER

```
// Data dari app.js:
// { sudahLogin: true }

<% if (sudahLogin) { %>
  <p>Status: Anda sudah masuk ke sistem.</p>
<% } else { %>
  <p>Silakan masuk terlebih dahulu.</p>
<% } %>
```

HASIL HTML YANG DITERIMA PERAMBAN

```
<p>Status: Anda sudah masuk ke sistem.</p>
```

Hanya cabang yang bernilai benar (`true`) yang dikirim menjadi HTML ke peramban.

SIMULASI INTERAKTIF • PROSES RENDER EJS

Ilustrasi perubahan kode EJS menjadi HTML

1. Variabel (<%= %>) | 2. Percabangan (if-else) | **3. Pengulangan (loop)**

KODE DI VIEWS/INDEX.EJS + DATA SERVER

```
// Data dari app.js:
// { daftarMenu: ['Beranda', 'Produk', 'Kontak'] }

<ul>
  <% daftarMenu.forEach(item => { %>
    <li><%= item %></li>
  <% }) %>
</ul>
```

HASIL HTML YANG DITERIMA PERAMBAN

```
<ul>
  <li>Beranda</li>
  <li>Produk</li>
  <li>Kontak</li>
</ul>
```

Pengulangan `forEach` membentuk elemen `<li>` sesuai jumlah data di dalam array.

PERSIAPAN LAB 3 • ALUR DATA

Pasangan kode di `app.js` dan `views/index.ejs`

Pastikan nama properti objek yang dikirim dari `app.js` sama persis dengan nama variabel yang dipanggil di dalam berkas EJS:

1. PENGIRIM DATA (SERVER)

Berkas `app.js`

```
app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  // Data seolah-olah dari database
  const dataUser = {
    namaPengguna: 'Peserta Pelatihan'
  };
  res.render('index', dataUser);
});
```

Objek `dataUser` memiliki properti bernama `namaPengguna`.

2. PENERIMA DATA (TEMPLATE)

Berkas `views/index.ejs`

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Aplikasi Node Saya</title>
</head>
<body>
  <h1>Selamat Datang, <%= namaPengguna %>!</h1>
  <p>Ini adalah aplikasi pertama menggunakan EJS.</p>
</body>
</html>
```

Tag `<%= namaPengguna %>` langsung menampilkan isi properti tersebut.

PERSIAPAN LAB 3 • KETELITIAN KODE

Tiga hal yang perlu diperhatikan saat praktik

Agar proses *render* berjalan lancar tanpa pesan galat, perhatikan tiga aturan berikut:

01 NAMA FOLDER

Gunakan nama `views`

Secara bawaan, Express mencari berkas template di dalam folder bernama `views` (jamak, menggunakan huruf **s** di akhir).

> Letakkan berkas di `views/index.ejs`, bukan di luar folder.

02 HURUF BESAR & KECIL

Penulisan variabel harus sama

JavaScript membedakan huruf besar dan kecil (*case-sensitive*). Penulisan `namaPengguna` di `app.js` dan `index.ejs` harus identik.

> Huruf **P** kapital pada `namaPengguna` jangan sampai terlewat.

03 PEMBARUAN SERVER

Restart server setelah ubah `app.js`

Setiap kali mengubah kode di `app.js`, matikan proses server dengan `Ctrl + C` lalu jalankan kembali `node app.js`.

> Setelah server menyala ulang, segarkan halaman di peramban.

SESI PRAKTIK • LAB 3

Saatnya praktik: merender data dinamis ke halaman EJS

Ikuti langkah **Lab 3: Implementasi EJS** pada proyek `web-app-saya`:

1. Pasang paket EJS melalui terminal: `npm install ejs`
2. Tambahkan konfigurasi `app.set('view engine', 'ejs');` di dalam `app.js`.
3. Buat folder `views/` dan berkas `views/index.ejs` berisi judul `<h1>Selamat Datang, <%= namaPengguna %>!</h1>`.
4. Perbarui rute `/` di `app.js` untuk mengirim data ke EJS:

```
app.get('/', (req, res) => {
  const dataUser = { namaPengguna: 'Peserta Pelatihan' };
  res.render('index', dataUser);
});
```

Matikan server dengan `Ctrl + C`, jalankan ulang `node app.js`, lalu segarkan `http://localhost:3000`.

TARGET LAB 3

Variabel tampil di halaman HTML

Peramban menampilkan judul **Selamat Datang, Peserta Pelatihan!** yang nilainya dikirim dari objek `dataUser` di server.

SETELAH LAB 3 SELESAI

Kita akan lanjut ke **Materi 4: Integrasi Bootstrap 5** untuk menata halaman EJS menggunakan sistem *grid* dan komponen antarmuka modern.